CPSC 304 Project Cover Page

Milestone #: 2
Date: 22nd October, 2025
Group Number: 30

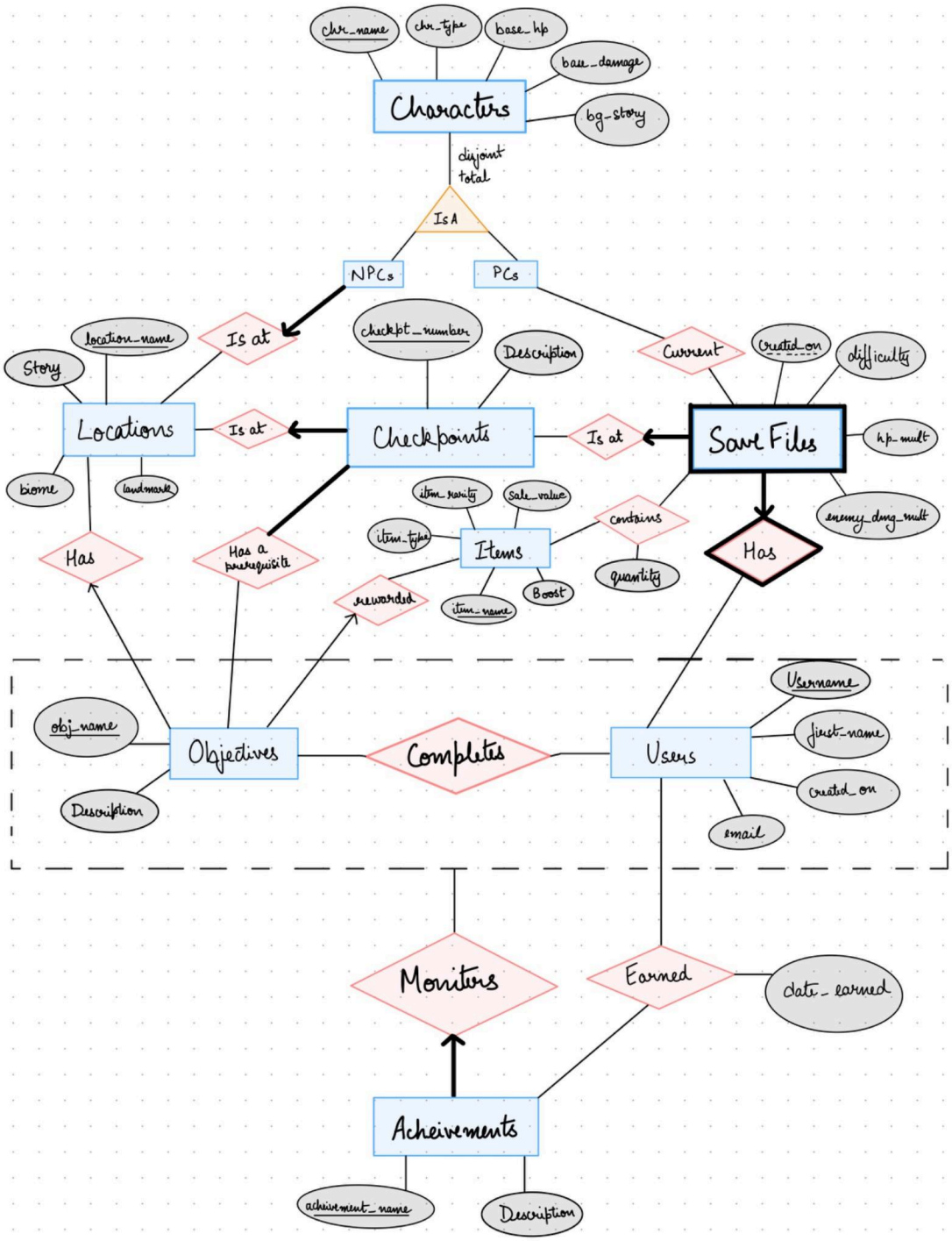
Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Ketan Saraf	71049779	f4t4q	saraf.ketan1108@gmail.com
Samarth Kapila	77825172	t5c4f	samarthkapila673@gmail.com
Norinne Norinne	46986816	i7a2e	norinne.arora@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)
In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

1. Summary

Our project is about creating a relational database for a role-playing video game. It stores user progress, achievements, game locations, items, and characters. This database can support the backend of modern games by saving player progress, tracking objectives, and connecting with gaming services like PlayStation.

2. ER Diagram



As suggested by our TA, we made the change to make created_on the partial key for Save Files since it is a weak entity. We added attributes in characters, locations, users and items to have additional functional dependencies other than just primary keys. We

also changed the names of certain attributes that were duplicated or were reserved keywords to help with readability.

3. Relational Schemas

The following are the relational schemas that we derived from our diagram

1. Users (username: char[20], first_name: char[20], created_on: date, email: varchar)
    a. Primary key: username
    b. Candidate key: username, email
    c. Not null: email
    d. Unique: email
    e. Foreign key: none

2. Objectives (obj_name: char[20], description: varchar, location_name: char[20], item_name: char[20])
    a. Primary key: obj_name
    b. Candidate key: obj_name
    c. Not null: none
    d. Unique: none
    e. Foreign key: location_name (references locations), item_name (references items)

3. Items (item_name: char[20], item_type: char[10], boost: int, item_rarity: char[20], sale_value: int)
    a. Primary Key: item_name
    b. Candidate Key: item_name
    c. Not null: item_rarity, item_type
    d. Unique: none
    e. Foreign key: none

4. Location (location_name: char[20], story: varchar, biome: char[10], landmark: char[20])
    a. Primary key: location_name
    b. Candidate key: location_name, (biome, landmark)
    c. Not null: biome, landmark
    d. Unique: (biome, landmark) (their combination must be unique)
    e. Foreign key: none

5. NPCs_IsAt(chr_name: char[20], chr_type: char[20], base_hp: int, base_damage: int, bg_story: char[20], location_name: char[20])
    a. Primary key: chr_name
    b. Candidate key: chr_name
    c. Not null: base_hp, base_damage, chr_type
    d. Unique: none
    e. Foreign key: location_name

6. PCs(chr_name: char[20], chr_type: char[20], base_hp: int, base_damage: int, bg_story: char[20])
    a. Primary key: chr_name
    b. Candidate key: chr_name
    c. Not null: base_hp, base_damage, chr_type
    d. Unique: none
    e. Foreign key: none

NPCs and PCs are IsA with Characters. In this case, we decides to opt for method 3 as specified in the slides to make the relations. It made the most sense in our case because the relationship is total and disjoint, and the superclass, Characters, is not in a relationship with any other entity. Hence, this was the best way to enforce the total disjoint aspect and minimise duplication.

7. Checkpoints_IsAt (checkpt_number: int, description: varchar, location_name: char[20])
    a. Primary key: checkpt_number
    b. Candidate key: checkpt_number
    c. Not null: location_name
    d. Unique: none
    e. Foreign key: location_name (references Location)

8. SaveFiles_IsAt (username: char[20], created_on: date, difficulty: char[20], enemy_dmg_mult: decimal, xp_gain_mult: decimal, checkpt_number: int)
    a. Primary key: (username, created_on)
    b. Candidate key: (username, created_on)
    c. Not null: difficulty, enemy_dmg_mult, xp_gain_mult, checkpt_number
    d. Unique: none
    e. Foreign key: username (references Users)

9. Achievements_monitors (achievement_name: char[20], description: varchar, username: char[20], obj_name: char[40])
    a. Primary key: achievement_name
    b. Candidate key: achievement_name
    c. Not null: description, username, obj_name
    d. Unique: none
    e. Foreign key: username (references Users), obj_name (references Objectives)

10. Completes(username: char[20], obj_name: char[20])
    a. Primary key: (username, obj_name)
    b. Candidate key: (username, obj_name)
    c. Not null: username, obj_name
    d. Unique: none
    e. Foreign key: username (references Users), obj_name (references Objectives)

11. Earned (achievement_name: char[20], username: char[20], date: date)
    a. Primary key: (achievement_name, username)
    b. Candidate key: (achievement_name, username)
    c. Not null: achievement_name, username, date
    d. Unique: none
    e. Foreign key: achievement_name (references Achievements_monitors), username (references Users)

12. Current (username: char[20], created_on: date, chr_name: char[20])
    a. Primary key: (username, created_on, chr_name)
    b. Candidate key: (username, created_on, chr_name)
    c. Not null: none
    d. Unique: none
    e. Foreign key: (username, created_on) (references SaveFiles_IsAt), chr_name (references PCs)

13. Contains (username: char[20], created_on: date, item_name: char[20], quantity: int)
    a. Primary key: (username, created_on, item_name)
    b. Candidate key: (username, created_on, item_name)
    c. Not null: none
    d. Unique: none

    e. Foreign key: (username, created_on) (references SaveFiles_IsAt), item_name (references Items)

14. HasAPrerequisite(checkpt_number: int, obj_name: char[20])
    a. Primary key: (checkpt_number, obj_name)
    b. Candidate key: (checkpt_number, obj_name)
    c. Not null: none
    d. Unique: none
    e. Foreign key: checkpt_number (references Checkpoints_IsAt), obj_name (references Objectives)

Note: This is a Many-to-many relationship with participation so we need assertion to model this.

4. Functional dependencies

1. username → first_name, created_on, email
2. email → username, first_name, created_on
3. obj_name → description, location_name, item_name
4. item_name → item_type, boost, item_rarity, sale_value
5. item_rarity → sale_value
6. item_type, item_rarity → boost
7. location_name → story, biome, landmark
8. biome, landmark → location_name, story
9. chr_name → chr_type, base_hp, base_damage, bg_story, location_name
10. chr_type → base_hp, base_damage
11. chr_name → chr_type, base_hp, base_damage, bg_story
12. chr_type → base_hp, base_damage
13. checkpt_number → description, location_name
14. username, created_on → difficulty, enemy_dmg_mult, xp_gain_mult
15. difficulty → enemy_dmg_mult, xp_gain_mult
16. achievement_name → description, username, obj_name
17. achievement_name, username → date

5. Normalization

- The FD, email → username is in BCNF as email is a candidate key in the table Users.
- The FD, chr_type → base_hp, base_damage is not in BCNF as the closure of chr_type is {base_hp,base_damage, chr_type} which is not all the attributes in the table PCs and NPCs. We break down our tables, PCs and NPCs on this FD.
    - PC is broken down into two tables(both of them are now in BCNF)

        - PCs
        - CharTypes
    - NPCs is broken down into two tables(both of them are now in BCNF)
        - NPCs
        - CharTypes
    - CharTypes includes both base_hp & base_damage
    - Note: We have only kept 1 CharTypes table because NPCs and PCs are the IsA of Characters in our relational model. If that's not allowed in BCNF, we would make it 2 same tables but with the same info.
- The FD, item_rarity → sale_value is not in BCNF as the closure of the item_rarity is {sale_value, item_rarity}. So, we split on this FD and obtain two tables, one with item_rarity and sale_value as attributes and the other one with item_rarity and the remaining attributes of Items. The newly obtained second relation is still not in BCNF due to the relation item_type, item_rarity → boost as item_type, item_rarity is not a candidate key. Splitting on this FD, we are left with three final relations for our initial Items table - ItemSaleValue, ItemBoost and Items, which are shown below.
- The FD, difficulty → enemy_dmg_mult, xp_gain_mult is not in BCNF as difficulty is not a candidate key in the SaveFiles_IsAt table so we split this table on this FD. This resulted in two tables (both of them in BCNF) - SaveFiles_IsAt and Difficulty as shown below.

FINAL SCHEMA:

CharTypes (chr_type: char[20], base_hp: int, base_damage: int)

- Primary key: chr_type
- Candidate key: chr_type
- Not null: base_hp, base_damage
- Unique: none
- Foreign key: none

ItemSaleValue (item_rarity: char[20], sale_value: int)

- Primary key: item_rarity

- Candidate key: item_rarity
- Not null: sale_value
- Unique: none
- Foreign key: none

Boost (item_rarity: char[20], item_type: char[20], boost: int)

- Primary key: (item_rarity, item_type)
- Candidate key: (item_rarity, item_type)
- Not null: boost
- Unique: none
- Foreign key: none

Difficulty (difficulty: char[20], hp_mult: int, enemy_dmg_mult: int)

- Primary key: difficulty
- Candidate key: difficulty
- Not null: hp_mult, enemy_dmg_mult
- Unique: none
- Foreign key: none

Users (username: char[20], first_name: char[20], created_on: date, email: varchar[255])

- Primary key: username
- Candidate key: username, email
- Not null: email

- Unique: email
- Foreign key: none

Objectives (obj_name: char[20], description: varchar[4000], location_name: char[20], item_name: char[20])

- Primary key: obj_name
- Candidate key: obj_name
- Not null: none
- Unique: none
- Foreign key: location_name (references Locations), item_name (references Items)

Items (item_name: char[20], item_rarity: char[20], item_type: char[20])

- Primary key: item_name
- Candidate key: item_name
- Not null: item_rarity, item_type
- Unique: none
- Foreign key: item_rarity (references ItemSaleValue), (item_rarity, item_type) (references ItemBoost)

Locations (location_name: char[20], story: varchar[4000], biome: char[10], landmark: char[20])

- Primary key: location_name
- Candidate key: location_name, (biome, landmark)
- Not null: biome, landmark
- Unique: (biome, landmark)
- Foreign key: none

NPCs_IsAt (chr_name: char[20], chr_type: char[20], bg_story: char[20], location_name: char[20])

- Primary key: chr_name
- Candidate key: chr_name
- Not null: chr_type
- Unique: none
- Foreign key: location_name (references Locations), chr_type (references CharTypes)

PCs (chr_name: char[20], chr_type: char[20], bg_story: char[20])

- Primary key: chr_name
- Candidate key: chr_name
- Not null: chr_type
- Unique: none
- Foreign key: chr_type (references CharTypes)

Checkpoints_IsAt (checkpt_number: int, description: varchar[4000], location_name: char[20])

- Primary key: checkpt_number
- Candidate key: checkpt_number
- Not null: location_name
- Unique: none
- Foreign key: location_name (references Locations)

SaveFiles_IsAt (username: char[20], created_on: date, difficulty: char[20])

- Primary key: (username, created_on)

- Candidate key: (username, created_on)
- Not null: difficulty
- Unique: none
- Foreign key: username (references Users), difficulty (references Difficulty)

Achievements_monitors (achievement_name: char[20], description: varchar, username: char[20], obj_name: char[20])

- Primary key: achievement_name
- Candidate key: achievement_name
- Not null: description, username, obj_name
- Unique: none
- Foreign key: username (references Users), obj_name (references Objectives)

Completes (username: char[20], obj_name: char[20])

- Primary key: (username, obj_name)
- Candidate key: (username, obj_name)
- Not null: username, obj_name
- Unique: none
- Foreign key: username (references Users), obj_name (references Objectives)

Earned (achievement_name: char[20], username: char[20], date_earned: date)

- Primary key: (achievement_name, username)
- Candidate key: (achievement_name, username)
- Not null: achievement_name, username, date_earned

- Unique: none
- Foreign key: achievement_name (references Achievements_monitors), username (references Users)

Current (username: char[20], created_on: date, chr_name: char[20])

- Primary key: (username, created_on, chr_name)
- Candidate key: (username, created_on, chr_name)
- Not null: none
- Unique: none
- Foreign key: (username, created_on) (references SaveFiles_IsAt), chr_name (references PCs)

Contains (username: char[20], created_on: date, item_name: char[20], quantity: int)

- Primary key: (username, created_on, item_name)
- Candidate key: (username, created_on, item_name)
- Not null: none
- Unique: none
- Foreign key: (username, created_on) (references SaveFiles_IsAt), item_name (references Items)

HasAPrerequisite (checkpt_number: int, obj_name: char[20])

- Primary key: (checkpt_number, obj_name)
- Candidate key: (checkpt_number, obj_name)
- Not null: none

- Unique: none
- Foreign key: checkpt_number (references Checkpoints_IsAt), obj_name (references Objectives)

AI Acknowledgement

Yes, we used ChatGPT to format our final schema, and the conversation link is below: https://chatgpt.com/share/68f98f53-a5f0-800a-87cc-45c00020ba91